

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501
NAME: M.NITHYASHREENARAYANI
REGNO: 192521416

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

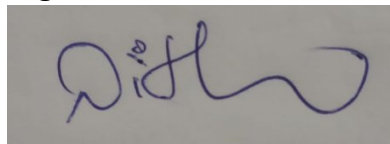
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I M.NITHYASHREENARAYANI / 192521416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

DATABASE MANAGEMENT SYSTEMS

Unit: Relational Database Design & Normalization

Model Answers — Functional Dependencies, Normal Forms (1NF–5NF), Keys & Decomposition

Total: 10 Questions | 50 Marks | Bloom's Levels: BL2 (Understand), BL3 (Apply)

Q1. Functional Dependencies & Candidate Keys — STUDENT_COURSE [5 Marks]

Relation: STUDENT_COURSE (SID, SName, CID, CName, Instructor, Grade)

Step 1: Understand the Attributes

SID – Student roll number, SName – Student name, CID – Course ID, CName – Course name, Instructor – Faculty teaching the course, Grade – Grade obtained by a student in a course.

Step 2: Identify the Functional Dependencies (FDs)

#	Functional Dependency	Justification
1	$SID \rightarrow SName$	Each student roll number identifies exactly one student name.
2	$CID \rightarrow CName$	Each course code identifies exactly one course name.
3	$CID \rightarrow Instructor$	Each course is assumed to be taught by one instructor.
4	$SID, CID \rightarrow Grade$	A grade exists only for a specific (student, course) combination — a student can get different grades in different courses.

Step 3: Determine the Candidate Key

Compute the closure of the composite attribute set {SID, CID}:

```
{SID, CID}+ = {SID, CID}
           ∪ {SName}    (using  $SID \rightarrow SName$ )
           ∪ {CName, Instructor} (using  $CID \rightarrow CName, Instructor$ )
           ∪ {Grade}    (using  $SID, CID \rightarrow Grade$ )
           = {SID, CID, SName, CName, Instructor, Grade} = all attributes of R
```

Since {SID, CID}⁺ equals the full attribute set, {SID, CID} is a super key. No proper subset ({SID} or {CID} alone) determines all attributes, so it is minimal.

Candidate Key = {SID, CID} (Composite / Primary Key)

Step 4: Prime and Non-Prime Attributes

- Prime attributes (part of the key): SID, CID
- Non-prime attributes: SName, CName, Instructor, Grade

Step 5: Normalization Observation

$SID \rightarrow SName$ and $CID \rightarrow CName$, Instructor are partial dependencies on the composite key $\{SID, CID\}$ ($SName$ depends only on part SID , and $CName/Instructor$ depend only on part CID). Hence $STUDENT_COURSE$ is in 1NF but violates 2NF. It should be decomposed into:

- $STUDENT (SID, SName)$
- $COURSE (CID, CName, Instructor)$
- $ENROLLMENT (SID, CID, Grade)$

Q2. Converting an Unnormalized Relation (UNF) to 1NF [5 Marks]

Step 1: What is an Unnormalized Relation (UNF)?

A relation is in UNF (also called NF^2 /nested form) when a cell can hold multiple values, a repeating group, or a nested table instead of a single atomic value. 1NF requires: (a) every attribute contains only atomic (indivisible) values, and (b) there are no repeating groups / multi-valued columns.

Step 2: Example Unnormalized Relation

Consider a relation $STUDENT_UNF$ that stores a student and the multiple courses they are enrolled in in a single row:

SID	SName	Courses (CID, CName)
S1	Aditi	(C1, DBMS), (C2, OS)
S2	Rahul	(C3, Networks)

Here the "Courses" column violates 1NF because it holds a set/list of values (a repeating group) instead of one atomic value.

Step 3: Justification for Why This Violates 1NF

- Atomicity is violated: a single cell holds two course entries for S1.
- It cannot be queried easily with standard SQL (e.g., "find all students of C2" requires parsing the cell).
- It leads to update anomalies — editing one course inside the group risks corrupting the others.

Step 4: Conversion Process to 1NF

1. Identify the repeating/multi-valued attribute (here, the Courses group containing CID and CName).
2. Flatten the repeating group by creating one row for every (student, course) pair — this removes nested values.
3. Ensure the primary key is redefined so each row is unique — here the key becomes the composite (SID, CID), since SID alone no longer identifies a row uniquely.
4. Verify every column now stores a single atomic value.

Step 5: Resulting 1NF Relation

SID	SName	CID	CName
S1	Aditi	C1	DBMS

SID	SName	CID	CName
S1	Aditi	C2	OS
S2	Rahul	C3	Networks

This relation, STUDENT_1NF (SID, SName, CID, CName), now satisfies 1NF: every cell holds a single atomic value and there are no repeating groups. (It still has redundancy — SName repeats — which is resolved in 2NF/3NF, covered in later questions.)

Q3. 2NF Decomposition — R(A,B,C,D,E) with FDs $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$ [5 Marks]

Step 1: Given

R(A, B, C, D, E)

FD1: $A \rightarrow BC$

FD2: $BC \rightarrow D$

FD3: $D \rightarrow E$

Step 2: Find the Candidate Key — Compute Closures

$A^+ = \{A\}$

$\cup \{B, C\}$ ($A \rightarrow BC$)

$\cup \{D\}$ ($BC \rightarrow D$, since B,C now in closure)

$\cup \{E\}$ ($D \rightarrow E$)

$A^+ = \{A, B, C, D, E\} = R$

Since $A^+ = R$ and no other single/derived attribute set is smaller, A is the sole candidate key of R.

Candidate Key = {A} (a single attribute)

Step 3: Check for Partial Dependency (2NF Test)

2NF is violated only when a non-prime attribute depends on a proper subset of a composite candidate key. Here the candidate key {A} is a single attribute, so it has no proper non-empty subset — a partial dependency is therefore structurally impossible.

Conclusion: R(A,B,C,D,E) is automatically in 2NF as given, because its only candidate key is non-composite. (2NF only becomes a real constraint when the key has two or more attributes, as in Q1's {SID, CID}.)

Step 4: Further Observation — Transitive Dependency (for completeness)

Although 2NF is already satisfied, note that $A \rightarrow BC \rightarrow D$ and $D \rightarrow E$ are chained (transitive) dependencies: A determines D and E only indirectly through B,C and D. This does not break 2NF, but it will break 3NF (Question 4 demonstrates the equivalent transitive-dependency decomposition). For illustration, decomposing along the functional-dependency chain gives:

Decomposed Relation	Retained FD	Key
R1(A, B, C)	$A \rightarrow BC$	A
R2(B, C, D)	$BC \rightarrow D$	BC

Decomposed Relation	Retained FD	Key
R3(D, E)	$D \rightarrow E$	D

This decomposition is lossless (each piece can be rejoined on the common attributes A, BC, D respectively) and removes all transitive dependencies, taking the design to 3NF/BCNF.

Q4. 3NF Decomposition — EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) [5 Marks]

Step 1: Given Functional Dependencies

#	FD	Meaning
1	$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$	Each employee has one name and belongs to one department.
2	$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$	Each department has one name and one manager.

Step 2: Candidate Key

$\text{EmpID}^+ = \{\text{EmpID}, \text{EmpName}, \text{DeptID}\}$
 $\cup \{\text{DeptName}, \text{ManagerID}\}$ (via $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$)
 $\text{EmpID}^+ = \{\text{EmpID}, \text{EmpName}, \text{DeptID}, \text{DeptName}, \text{ManagerID}\} = R$

Candidate Key = {EmpID}.

Step 3: Detect the Transitive Dependency

A transitive dependency exists when: $\text{Key} \rightarrow X \rightarrow Y$, where X is not a key and Y is a non-prime attribute. Here:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

DeptID is a non-key attribute of EMPLOYEE, yet it determines DeptName and ManagerID. So DeptName and ManagerID depend on the key EmpID only transitively (through DeptID) — this violates 3NF.

Step 4: 3NF Decomposition Rule Applied

For each transitive FD ($X \rightarrow Y$ where X is not a super key), remove Y from the original relation and place $X \rightarrow Y$ in a new relation, keeping X as a foreign key link.

Decomposed Relation	Attributes	Key	Foreign Key
EMPLOYEE	EmpID, EmpName, DeptID	EmpID	$\text{DeptID} \rightarrow \text{DEPARTMENT.DeptID}$
DEPARTMENT	DeptID, DeptName, ManagerID	DeptID	—

Step 5: Verification

- Lossless join: $\text{EMPLOYEE} \bowtie \text{DEPARTMENT}$ (on DeptID) reconstructs the original EMPLOYEE relation exactly, since DeptID is a key of DEPARTMENT.
- Dependency preserving: both original FDs ($\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$) and ($\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$) are enforceable within the two new relations.

- 3NF check: in EMPLOYEE every non-prime attribute (EmpName, DeptID) depends directly on the key EmpID; in DEPARTMENT every non-prime attribute (DeptName, ManagerID) depends directly on the key DeptID. No transitive dependency remains.

Q5. BCNF Verification & Decomposition — $R(A,B,C,D)$, FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ [5 Marks]

Step 1: Find All Candidate Keys

$(AB)^+ = \{A,B\} \cup \{C\}$ ($AB \rightarrow C$) $\cup \{D\}$ ($C \rightarrow D$) = $\{A,B,C,D\} = R \rightarrow AB$ is a candidate key
 $(BC)^+ = \{B,C\} \cup \{D\}$ ($C \rightarrow D$) $\cup \{A\}$ ($D \rightarrow A$) = $\{A,B,C,D\} = R \rightarrow BC$ is a candidate key
 $(BD)^+ = \{B,D\} \cup \{A\}$ ($D \rightarrow A$) = $\{A,B,D\} \neq R$ (missing C) $\rightarrow$ not a key
 $C^+ = \{C,D,A\} \neq R$ (missing B) $\rightarrow$ not a key alone

Candidate Keys = {AB, BC} $\rightarrow$ Prime attributes: A, B, C. Non-prime attribute: D.

Step 2: BCNF Test — Every Determinant Must Be a Super Key

FD	Is LHS a super key?	BCNF status
$AB \rightarrow C$	Yes (AB is a candidate key)	Satisfies BCNF
$C \rightarrow D$	No — C alone is not a super key	VIOLATES BCNF
$D \rightarrow A$	No — D alone is not a super key	VIOLATES BCNF

Conclusion: $R(A,B,C,D)$ is NOT in BCNF because $C \rightarrow D$ (and $D \rightarrow A$) have determinants that are not super keys.

Step 3: BCNF Decomposition (Repeated Application of the Violating FD)

Rule: for a violating FD $X \rightarrow Y$, decompose R into $R_1(X, Y)$ and $R_2(R - Y, X)$.

Iteration 1 — apply $C \rightarrow D$:

- $R_1(C, D)$ — since $C \rightarrow D$, key of R_1 is C.
- $R_2(A, B, C)$ — the remainder, $R - \{D\} \cup \{C\}$.

Note: because $D \rightarrow A$ also holds and $C \rightarrow D$, transitively $C \rightarrow A$ holds too — this FD carries over into R_2 .

Check $R_2(A, B, C)$ for BCNF:

Candidate key of R_2 : AB ($AB \rightarrow C$, and no smaller set determines all of A,B,C)
 FD carried over: $C \rightarrow A$ (derived: $C \rightarrow D \rightarrow A$, restricted to attributes of R_2)
 Is C a super key of R_2 ? No — C alone does not determine B.
 $\rightarrow R_2(A,B,C)$ still violates BCNF.

Iteration 2 — apply $C \rightarrow A$ on $R_2(A,B,C)$:

- $R_{2a}(A, C)$ — since $C \rightarrow A$, key of R_{2a} is C.
- $R_{2b}(B, C)$ — the remainder; no non-trivial FD holds here, so the whole set (B,C) is the key.

Step 4: Final BCNF Decomposition

Relation	Attributes	Key	FD Enforced
R1	C, D	C	$C \rightarrow D$
R2a	A, C	C	$C \rightarrow A$
R2b	B, C	B, C (whole key)	— (all-key relation)

Every determinant in R1, R2a, and R2b is now a super key of its own relation, so all three relations are in BCNF. The decomposition is lossless because C (a key of R1 and R2a) is common across the pieces, allowing exact reconstruction via natural join.

Q6. Verifying Lossless-Join Decomposition Using the Tabular (Chase) Algorithm [5 Marks]

Step 1: Given Relation and Decomposition

$R(A, B, C, D)$ FDs: $A \rightarrow B$, $B \rightarrow C$

Decomposed into:

$R1(A, B)$

$R2(B, C)$

$R3(A, D)$

We must verify whether $\{R1, R2, R3\}$ is a lossless-join decomposition of R using the Chase (tableau) algorithm.

Step 2: Construct the Initial Tableau

Draw one row per decomposed relation and one column per attribute of R . Fill a cell with the unsubscripted symbol a_j if that relation contains the attribute; otherwise fill it with a subscripted symbol b_{ij} (row i , attribute j).

Relation	A	B	C	D
$R1(A,B)$	a_1	a_2	b_{13}	b_{14}
$R2(B,C)$	b_{21}	a_2	a_3	b_{24}
$R3(A,D)$	a_1	b_{32}	b_{33}	a_4

Step 3: Apply the FDs to Equate Rows (Chase Steps)

Apply $A \rightarrow B$:

Row1 and Row3 both have $A = a_1$ (same value), so by $A \rightarrow B$ their B -values must also be equal. Row1 has $B = a_2$ (unsubscripted); set Row3's $B = b_{32}$ to a_2 .

Updated Row3 = (a_1, a_2, b_{33}, a_4)

Apply $B \rightarrow C$:

Now Row1, Row2, and Row3 all have $B = a_2$. By $B \rightarrow C$, all their C -values must be equal. Row2 already has $C = a_3$ (unsubscripted); set Row1's $C = b_{13} \rightarrow a_3$ and Row3's $C = b_{33} \rightarrow a_3$.

Updated Row1 = (a_1, a_2, a_3, b_{14})

Updated Row3 = (a_1, a_2, a_3, a_4)

Step 4: Final Tableau

Relation	A	B	C	D
R1(A,B)	a1	a2	a3	b14
R2(B,C)	b21	a2	a3	b24
R3(A,D)	a1	a2	a3	a4

Step 5: Conclusion

Row 3 has become (a1, a2, a3, a4) — a row containing only unsubscripted symbols across every column. According to the Chase algorithm, the presence of an all-"a" row proves that the join of R1, R2, R3 always reconstructs exactly the original relation R with no spurious tuples.

Result: {R1(A,B), R2(B,C), R3(A,D)} is a LOSSLESS-JOIN decomposition of R(A,B,C,D).

Q7. Multi-Valued Dependencies & 4NF — TEACHER(TID, Subject, Hobby) [5 Marks]

Step 1: The Scenario

A teacher can teach multiple subjects and independently pursue multiple hobbies. Neither set is related to the other — e.g., a teacher's subjects do not influence which hobbies they have. A sample instance:

TID	Subject	Hobby
T1	Maths	Chess
T1	Maths	Painting
T1	Physics	Chess
T1	Physics	Painting

Notice every combination of T1's subjects with T1's hobbies must appear — this cross-product pattern is the signature of a multi-valued dependency.

Step 2: Identify the Multi-Valued Dependencies (MVDs)

TID $\twoheadrightarrow$ Subject

TID $\twoheadrightarrow$ Hobby

Read as: "for a given TID, the set of Subjects is independent of the set of Hobbies." Both are MVDs because Subject and Hobby are two independent multi-valued facts about the same TID, forced together into one row only because they share the key TID.

Step 3: Why This Violates 4NF

A relation is in 4NF only if, whenever a non-trivial MVD $X \twoheadrightarrow Y$ holds, X is a super key. Here TID is the only key candidate for the full (TID, Subject, Hobby) combination, but TID alone does not determine Subject or Hobby (they are multi-valued, not single-valued), so TID $\twoheadrightarrow$ Subject and TID $\twoheadrightarrow$ Hobby are non-trivial MVDs whose determinant TID is not a super key of the 3-attribute relation. This causes redundancy — every subject is repeated once per hobby and vice versa.

Step 4: Decompose to 4NF

Rule: for a non-trivial MVD $X \twoheadrightarrow Y$ in R , decompose R into $R_1(X, Y)$ and $R_2(X, R - X - Y)$.

Decomposed Relation	Attributes	Key
TEACHER_SUBJECT	TID, Subject	TID, Subject
TEACHER_HOBBY	TID, Hobby	TID, Hobby

Step 5: Resulting 4NF Instance

TID	Subject
T1	Maths
T1	Physics

TID	Hobby
T1	Chess
T1	Painting

Each independent multi-valued fact is now stored once, eliminating the redundant cross-product rows. Joining TEACHER_SUBJECT and TEACHER_HOBBY on TID losslessly reconstructs the original (though redundant) TEACHER relation.

Q8. Join Dependencies, Redundancy & 5NF (Project-Join Normal Form) [5 Marks]

Step 1: Concept — How Join Dependencies Cause Redundancy

A relation can be in BCNF and 4NF, yet still contain redundancy if it satisfies a join dependency (JD) that is not implied by its candidate key(s). A join dependency $JD(R_1, R_2, \dots, R_n)$ means R can always be reconstructed by joining projections $R_1, R_2, \dots, R_n$, but not by joining any two of them alone — the redundancy only becomes visible / removable through a three-way (or more) decomposition.

Step 2: Classic Example — SUPPLY(Supplier, Part, Project)

Business rule: "if supplier S supplies part P , and part P is used in project J , and supplier S supplies project J , then S supplies part P specifically for project J ." This is a cyclic, three-way constraint that cannot be expressed as an ordinary FD or MVD.

Supplier	Part	Project
S1	P1	J1
S1	P2	J1
S1	P1	J2
S2	P1	J1

Suppose we know independently that: S1 supplies P1 (to some project), P1 is used in J2, and S1 supplies J2. The business rule forces the row (S1, P1, J2) to exist — this tuple is redundant information that is fully implied by combining pairwise facts, yet a plain BCNF/4NF check would not catch it, because no single-attribute or MVD violation exists here.

Step 3: The Join Dependency

JD (SUPPLIER_PART, PART_PROJECT, SUPPLIER_PROJECT)
 $\equiv R = \pi\{S,P\}(R) \bowtie \pi\{P,J\}(R) \bowtie \pi\{S,J\}(R)$

This states that the original 3-attribute relation is exactly recoverable by joining its three binary projections — and, critically, no join of any two of the three projections alone reconstructs R without introducing spurious tuples. This is a non-trivial join dependency not implied by any candidate key, so SUPPLY violates 5NF (Project-Join Normal Form, PJNF).

Step 4: Decomposition to 5NF

Decomposed Relation	Attributes	Purpose
SUPPLIER_PART	Supplier, Part	Which parts each supplier can supply
PART_PROJECT	Part, Project	Which parts each project uses
SUPPLIER_PROJECT	Supplier, Project	Which projects each supplier serves

Step 5: Why This Is 5NF

- Each binary projection stores only a genuinely independent pairwise fact — no further lossless decomposition is possible without losing information.
- The original relation is reconstructed only via the full 3-way natural join of all three projections (matching the JD), and every join dependency present is implied by the candidate keys of the projections — satisfying the definition of 5NF.
- Redundant / inferable rows (like the forced S1-P1-J2 tuple) no longer need to be physically stored — they can be derived on demand by joining the three tables.

Q9. Anomalies in a Poorly Designed College Schema & Normalization to 3NF [5 Marks]

Step 1: The Poorly Designed (Unnormalized) Schema

COLLEGE (StudentID, StudentName, DeptID, DeptName, HOD,
CourseID, CourseName, Credits, Marks)

Sample data:

StudentID	StudentName	DeptID	DeptName	HOD	CourseID	CourseName	Marks
S1	Aditi	D1	CSE	Dr. Rao	C1	DBMS	85
S1	Aditi	D1	CSE	Dr. Rao	C2	OS	78
S2	Rahul	D1	CSE	Dr. Rao	C1	DBMS	90

Step 2: Identify the Anomalies

Anomaly Type	Example from the Table	Explanation
Insertion Anomaly	A new department (say D2, ECE, Dr. Iyer) cannot be added until at least one student from it enrolls	Department data is trapped inside student-course rows, so department facts can't exist

Anomaly Type	Example from the Table	Explanation
	in a course.	independently.
Deletion Anomaly	If S2 (the only student in DBMS besides S1) is deleted, and later S1's DBMS row is also removed, all knowledge that "Dr. Rao heads CSE" tied to that context could be unintentionally lost if it existed nowhere else.	Deleting a student's enrollment can accidentally wipe out department or course facts that happen to be represented only in that row.
Update Anomaly	If Dr. Rao is replaced as HOD, the value "Dr. Rao" must be updated in every row where DeptID = D1 (both of S1's rows and S2's row).	The same fact (HOD of D1) is repeated across many rows, risking inconsistency if not all copies are updated.

Step 3: Functional Dependencies Governing the Schema

FD	Meaning
StudentID → StudentName, DeptID	A student has one name and belongs to one department.
DeptID → DeptName, HOD	A department has one name and one head.
CourseID → CourseName, Credits	A course has one name and credit value.
StudentID, CourseID → Marks	Marks depend on the specific student-course combination.

Step 4: Normalize to 3NF

Remove partial dependencies (2NF) and transitive dependencies (3NF) by grouping attributes with their true determinant:

Relation	Attributes	Key	Foreign Keys
STUDENT	StudentID, StudentName, DeptID	StudentID	DeptID → DEPARTMENT
DEPARTMENT	DeptID, DeptName, HOD	DeptID	—
COURSE	CourseID, CourseName, Credits	CourseID	—
ENROLLMENT	StudentID, CourseID, Marks	StudentID, CourseID	StudentID → STUDENT, CourseID → COURSE

Step 5: How This Removes the Anomalies

- Insertion: a new department can now be inserted into DEPARTMENT independently of any student enrollment.
- Deletion: deleting a student's ENROLLMENT row no longer risks losing DEPARTMENT or COURSE facts, since those live in their own tables.
- Update: HOD is stored exactly once per DeptID in DEPARTMENT, so changing it requires a single update, not many.

Each relation is now in 3NF — every non-prime attribute depends on the whole key and nothing but the key.

Q10. Computing Attribute Closures & Determining Candidate Keys [5 Marks]

Step 1: Given

R(A, B, C, D, E)
FD1: $A \rightarrow B$
FD2: $B \rightarrow C$
FD3: $C \rightarrow D$
FD4: $D \rightarrow E$

Step 2: Closure Algorithm (Recap)

To compute X^+ (the closure of attribute set X): start with result = X; repeatedly scan every FD ($LHS \rightarrow RHS$); if $LHS \subseteq \text{result}$, add RHS to result; repeat until no more attributes can be added.

Step 3: Compute the Closure of Every Single Attribute

Attribute Set	Step-by-step Closure Computation	Closure (X^+)	= R ?
{A}	{A} $\rightarrow$ add B ($A \rightarrow B$) $\rightarrow$ add C ($B \rightarrow C$) $\rightarrow$ add D ($C \rightarrow D$) $\rightarrow$ add E ($D \rightarrow E$)	{A,B,C,D,E}	Yes
{B}	{B} $\rightarrow$ add C ($B \rightarrow C$) $\rightarrow$ add D ($C \rightarrow D$) $\rightarrow$ add E ($D \rightarrow E$)	{B,C,D,E}	No (missing A)
{C}	{C} $\rightarrow$ add D ($C \rightarrow D$) $\rightarrow$ add E ($D \rightarrow E$)	{C,D,E}	No
{D}	{D} $\rightarrow$ add E ($D \rightarrow E$)	{D,E}	No
{E}	{E} — no FD has E alone as LHS	{E}	No

Step 4: Determine the Candidate Key

Only $\{A\}^+$ equals the full attribute set $R = \{A,B,C,D,E\}$. Since a single attribute already yields a super key, and no attribute set is smaller than a single attribute, $\{A\}$ is minimal.

Candidate Key = {A} (the unique candidate key of R)

Step 5: Classify All Attributes

- Prime attribute: A (part of the only candidate key)
- Non-prime attributes: B, C, D, E — each depends on A only transitively, through the chain $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

This transitive chain also shows the relation is not in 3NF (B, C, D, E each depend on the key only indirectly) — the same removal technique demonstrated in Question 4 would apply here to reach 3NF: split into $R_1(A,B)$, $R_2(B,C)$, $R_3(C,D)$, $R_4(D,E)$.